# Group Project Mid-Submission Report

## Project Title

**Experiential Context Management: Learning Transferable Lessons from Counterfactual Agent Outcomes**

## Team Details

| Name | Roll Number / Email | Contribution |
|---|---|---|
| Sai Padakanti | psaiprasad.986@gmail.com | Project setup, GitHub repository, context-management pipeline |
| Ajit Yadav | ajitghz582@gmail.com | Literature review, project discussion, implementation support |
| Hitesh Kumar | hitu20128046@gmail.com | Literature review, dataset/evaluation support |
| Anil Setti | anilsetti591@gmail.com | Literature review, experiments/report support |

## GitHub Repository

Code repository:

```
https://github.com/sai986/experiential-context-management-agent
```

The repository contains the current implementation, controlled dataset files, pilot experiment script, saved raw trace, processed summary, and project documentation.

## 1. Problem Statement

Long-horizon LLM agents accumulate many observations, constraints, intermediate outputs, distractors, and failed attempts. Keeping everything in context increases token cost and may add noise. Compressing or removing information reduces context size but can delete important facts such as exact numeric values, negations, hard constraints, or delayed dependencies.

Our project studies whether a context-management agent can learn which information should be kept, compressed, archived, restored, or later retrieved. The key idea is to use counterfactual replay: when the task fails, we change an earlier context-management decision and check whether the downstream outcome improves. Only decisions with supported effects are converted into reusable context-management lessons.

## 2. Research Question

Can an experiential context manager improve long-horizon task performance by learning reusable context-management lessons from counterfactual replay, without retraining the frozen task-solving agent?

## 3. Current Progress

For the mid-submission checkpoint, we implemented a minimal controlled pilot. The current codebase includes:

• controlled synthetic task generation;
• deterministic evaluator;
• immutable context ledger;
• context-management actions for KEEP, COMPRESS, ARCHIVE, and RESTORE;

• full-context, fixed-compression, and non-learning-manager baselines;
• counterfactual replay by restoring original content after harmful compression;
• lesson creation from attributed failures;
• lesson-aware manager evaluation;
• saved raw traces and processed pilot results.

The current version intentionally uses a deterministic simulated task agent. This allows us to test the causal attribution and lesson loop without API-key dependencies or hosted-model nondeterminism. In the final version, this can be replaced or supplemented with a frozen hosted LLM.

## 4. Dataset Being Used

We use a controlled synthetic benchmark designed specifically for context-management failures. The dataset is generated by code in:

```
src/environment/task_generator.py
```

Generated dataset files:

```
data/controlled/train.jsonl
data/controlled/validation.jsonl
data/controlled/test.jsonl
```

Current task families:

| Task Family | Purpose | Example Failure |
|---|---|---|
| Hard-constraint preservation | Tests whether exact requirements survive compression | "Do not use cloud APIs" is compressed to "deployment has constraints" |
| Delayed dependency preservation | Tests whether earlier facts remain available when needed later | "Windows-only" is lost, so the final answer recommends Linux commands |

The pilot split currently contains:

| Split | Number of Tasks |
|---|---|
| Train | 20 |
| Validation | 5 |
| Test | 5 |

## 5. System Architecture Implemented

The implemented pilot follows this flow:

```
Controlled task
-> context ledger stores raw observations
-> context manager applies KEEP / COMPRESS / ARCHIVE / RESTORE
-> frozen deterministic task agent answers from managed context
-> evaluator scores answer
-> failed episodes trigger counterfactual replay
-> replay restores original compressed fact
-> improved outcome creates a reusable lesson
-> lesson-aware manager uses lessons on held-out tasks
```

Important files:

| Component | File |
|---|---|
| Task generation | `src/environment/task_generator.py` |
| Evaluation | `src/environment/evaluator.py` |
| Context ledger | `src/context/ledger.py` |
| Context managers | `src/context/manager.py` |
| Task agent | `src/agents/task_agent.py` |

| Component | File |
|---|---|
| Counterfactual replay | `src/attribution/replay.py` |
| Lesson store | `src/lessons/lesson_store.py` |
| Pilot experiment | `src/experiments/run_pilot.py` |

## 6. Baseline Experiments

We implemented three baseline conditions:

| Condition | Description |
|---|---|
| `full_context` | Gives all observations to the task agent without compression |
| `fixed_compression` | Compresses older context using a fixed strategy |
| `non_learning_manager` | Uses context-management rules but no learned lessons |

The proposed mid-submission method is:

| Condition | Description |
|---|---|
| `attributed_lesson_manager` | Uses lessons learned from counterfactual replay of previous failures |

## 7. Ablations

The current pilot supports these ablations:

| Ablation | Comparison | What It Tests |
|---|---|---|
| Full context vs fixed compression | `full_context` vs `fixed_compression` | Whether naive compression loses critical information |
| Non-learning vs lesson-aware manager | `non_learning_manager` vs `attributed_lesson_manager` | Whether learned context-management lessons help |
| Compression without attribution vs with attribution | `fixed_compression` / `non_learning_manager` vs `attributed_lesson_manager` | Whether replay-backed lessons prevent repeated failures |

Further ablations planned for final submission:

• no replay, naive reflection only;
• replay with single rollout vs repeated rollouts;
• with and without RETRIEVE;
• within-agent learning vs cross-agent transfer.

## 8. Pilot Results

The pilot was run using:

```
python -m src.experiments.run_pilot
```

Processed result file:

```
results/processed/pilot_summary.json
```

Current pilot results on the controlled test split:

| Condition | Test Tasks | Success Rate | Avg. Constraint Satisfaction | Avg. Active Context Tokens |
|---|---|---|---|---|
| `full_context` | 5 | 1.00 | 1.00 | 50.6 |

| Condition | Test Tasks | Success Rate | Avg. Constraint Satisfaction | Avg. Active Context Tokens |
|---|---|---|---|---|
| `fixed_compression` | 5 | 0.00 | 0.00 | 40.0 |
| `non_learning_manager` | 5 | 0.00 | 0.00 | 25.4 |
| `attributed_lesson_manager` | 5 | 1.00 | 1.00 | 32.4 |

Training replay summary:

| Metric | Value |
|---|---|
| Counterfactual training replays | 20 |
| Harmful replay cases found | 16 |
| Validated lessons stored | 2 |

The main observation from the pilot is that risky compression can remove critical facts, causing failures. Counterfactual replay identifies those harmful compression decisions, and the lesson-aware manager preserves similar critical information in later tasks.

These are controlled pilot results and should not be interpreted as final generalization claims.

## 9. Example Qualitative Case

Example raw fact:

```
The deployment must run offline and must not use any cloud API.
```

Risky compression:

```
The deployment has constraints.
```

Failure:

```
The task agent recommends a hosted cloud endpoint, violating the offline/no-cloud requirement.
```

Counterfactual intervention:

```
Restore the original raw fact in the active context.
```

Improved replay outcome:

```
The task agent recommends a local offline model with no cloud API.
```

Learned lesson:

```
Preserve critical constraints, negations, exact values, and delayed dependencies verbatim during
compression.
```

## 10. Challenges Faced

| Challenge | Impact |
|---|---|
| Scope is large | Full system includes memory, attribution, lessons, baselines, and transfer |
| Hosted LLM replay can be costly | Counterfactual replay multiplies model calls |
| Hosted models are stochastic | Same prompt may produce different outputs across runs |
| False causal claims are possible | Reflection alone can blame the wrong context decision |
| Cross-agent transfer is difficult | Lessons may help one agent but harm another |
| Time before mid-submission is short | We needed a minimal defensible pilot first |

## 11. Mitigation Plan

| Risk | Mitigation |
|---|---|
| Scope explosion | Start with two controlled task families and one replay intervention |
| Replay cost | Use controlled deterministic pilot first; add repeated hosted-model runs later |
| False attribution | Store a lesson only when counterfactual replay improves outcome |
| Dataset ambiguity | Use synthetic tasks with known critical facts and forbidden outputs |
| Negative transfer | Measure Agent-A to Agent-B transfer separately in final submission |
| Weak baselines | Compare against full context, fixed compression, and non-learning manager |

## 12. Timeline for Final Submission

Final submission deadline: **15 November 2026**

| Date Range | Planned Work |
|---|---|
| 31 Oct - 3 Nov | Clean code, add tests, expand controlled dataset |
| 4 Nov - 6 Nov | Add hosted LLM task-agent option and prompt versioning |
| 7 Nov - 9 Nov | Add RETRIEVE action and more replay interventions |
| 10 Nov - 11 Nov | Run full baseline and ablation experiments |
| 12 Nov - 13 Nov | Add Agent B and cross-agent transfer experiment |
| 14 Nov | Complete final report, figures, and error analysis |
| 15 Nov | Freeze repository and submit final artifacts |

## 13. Planned Final Experiments

The final submission will expand the pilot in the following directions:

1. Replace or supplement the simulated task agent with a frozen LLM.
2. Add repeated replay runs for stochastic models.
3. Add RETRIEVE and archive-to-keep interventions.
4. Expand controlled task families.
5. Add cross-agent transfer using Agent A and Agent B.
6. Report token cost, replay cost, attribution accuracy, and negative transfer.

## 14. Current Limitations

- The current task agent is deterministic and simulated.
- The test split is small.
- Current replay supports only restore-after-compression.
- RETRIEVE is part of the design but not yet implemented in the pilot.
- Results are from controlled tasks, not open-ended long-horizon benchmarks.
- Cross-agent transfer is planned for final submission, not completed yet.

## 15. Conclusion

The mid-submission implementation demonstrates the core project loop: context decisions are recorded, harmful compression can be identified through counterfactual replay, validated lessons are stored, and those lessons improve later controlled tasks under the same pilot setup. The final phase will focus on scaling the benchmark, adding hosted LLM agents, expanding ablations, and evaluating cross-agent transfer.